



NurvexThink

Software, built and published.

Products Experience — Design Specification

Public showcase + flexible admin: content AND presentation control

Date: 2026-07-04

Status: Approved (design phase)

Owner: Fatima Abdul Raheem (CEO)

Depth decision: Content + presentation control

Contents

1	Overview	2
1.1	Why this shape (from the 2026-07-03 admin-flexibility research)	2
1.2	Goals	2
1.3	Non-goals (deferred)	2
2	Public UX	2
2.1	/products — showcase grid	2
2.2	/products/[slug] — detail page	2
3	Admin UX (/admin/products)	3
4	Data model (Supabase)	3
4.1	product_categories (new)	3
4.2	products (revised from the 2026-06-30 site spec)	3
4.3	profiles (from the site spec, built here)	3
4.4	Indexes	3
4.5	RLS (default-deny, enabled on every table)	3
5	Auth & guarding	4
6	Performance	4
7	Testing	4
8	Build order (three PRs)	4

1 Overview

Build the **products experience** for the NurvexThink main site: a public product showcase (grid + real per-product detail pages) driven end-to-end by a flexible admin. The admin controls both **what** shows (content, media) and **how** it's arranged (order, pinning, categories, publish state) — with drafts and preview, so nothing goes live by accident.

1.1 Why this shape (from the 2026-07-03 admin-flexibility research)

We reviewed a reference admin (Fatima's portfolio) end-to-end. Its strengths — every admin field labeled with its public destination; instant content propagation — are adopted here. Its gaps define our differentiators:

Reference gap	Our answer
No ordering/pinning	<code>sort_order</code> + drag-to-reorder; featured pinning
Everything live on save	Draft → Published workflow + preview
Hardcoded category dropdown	Admin-managed categories table
No detail pages / SEO	<code>/products/[slug]</code> pages with per-product SEO
No live preview	Preview renders the real public page from draft data

1.2 Goals

- Public showcase that sells: filterable grid + rich detail pages linking to live apps.
- Admin can reshape the grid (order, featured, categories) without a deploy.
- Safe publishing: draft/preview/publish; nothing accidental.
- Foundation (schema, RLS, auth) reusable by blog + orders later.

1.3 Non-goals (deferred)

- Blog admin & orders inbox (separate specs; this builds their foundation).
- Payments, client accounts.
- Full site-builder (home hero/stats editing from admin) — revisit after products ship.

2 Public UX

2.1 `/products` — showcase grid

- Card: cover image, name, summary, category pill, up to ~4 tech chips, Featured badge.
- Filter pills from the categories table (+ “All”); empty categories hidden.
- Order: featured DESC, `sort_order` ASC — exactly what the admin arranged.
- Server Component; data via the Supabase server client; `revalidatePath` on admin mutations keeps it fresh without polling.

2.2 `/products/[slug]` — detail page

- Hero: cover image, name, one-line summary, category, CTA row (**Open live app**, **View repo** when set).

- **Body:** long description (Markdown rendered), tech stack chips, gallery grid, and a “**Request something like this**” CTA linking to `/order?ref=<slug>`.
- **SEO:** generate Metadata from `seo_description` (fallback: summary) + OG image (`og_image` fallback: cover). Published products only; drafts 404 publicly.

3 Admin UX (`/admin/products`)

- **List view:** cover thumb, name, category, status badge (Draft/Published), Featured star, **drag-to-reorder** (persists `sort_order`); filter by status/category.
- **Editor:** every field labeled with its public destination — e.g. “Summary (*product card*)”, “Long description (*detail page*)”, “SEO description (*search results*)”. Fields: name, slug (auto from name, editable), summary, long description (Markdown), category select, tech list, cover image upload, gallery uploads, live URL, repo URL, featured toggle, SEO description, OG image. Buttons: **Save draft** · **Preview** · **Publish** (and Unpublish on published items).
- **Preview:** admin-only route rendering the *actual* public detail-page component from draft data (`/admin/products/[id]/preview`).
- **Categories manager:** add / rename / delete / reorder. Delete blocked while products reference the category (reassign first).
- Uploads go to the `product-images` Storage bucket (public read, admin write).

4 Data model (Supabase)

4.1 `product_categories` (new)

`id uuid PK · name text · slug text unique · sort_order int · created_at`

4.2 `products` (revised from the 2026-06-30 site spec)

`id uuid PK · slug text unique · name text · summary text · description text (Markdown) · cover_image text · gallery text[] · category_id uuid FK (nullable) · tech text[] · live_url text · repo_url text · featured bool default false · sort_order int default 0 · status text check in ('draft', 'published') default 'draft' (replaces the earlier published bool) · seo_description text · og_image text · published_at timestamptz · created_at · updated_at`

4.3 `profiles` (from the site spec, built here)

`id uuid PK = auth.users.id · email · full_name · role text check in ('owner', 'admin') · created_at`

4.4 Indexes

`products(status)`, `products(featured)`, `products(category_id)`, `products(sort_order)`, `products(slug)`, `product_categories(slug)`, `profiles(role)` — every column an RLS policy or common filter touches.

4.5 RLS (default-deny, enabled on every table)

- `products`: public SELECT where `status = 'published'`; admins (via `profiles.role` in `('owner', 'admin')`) full access.
- `product_categories`: public SELECT; admins full access.
- `profiles`: user reads own row; admins read all; only owners change roles.

- Storage `product-images`: public read; write restricted to authenticated admins.

Migrations are SQL files committed under `supabase/migrations/` and applied to project `axbsghyqhhdaiylck` via the Supabase SQL editor or CLI.

5 Auth & guarding

- Supabase Auth email/password for the three admin accounts.
- `/admin/login` public; all other `/admin/*` routes guarded server-side (`@supabase/ssr` session + `profiles.role` check in the admin layout).
- `service_role` key never leaves the server; mutations run through Server Actions that re-verify the admin session.

6 Performance

- Server Components throughout; the admin's interactive pieces (drag list, editor form, uploads) are client leaves.
- Public pages cached; admin mutations call `revalidatePath('/products')` (+ the detail path), so publishing is instant without polling.
- `next/image` everywhere; drag-and-drop via native HTML5 DnD (no new dependency unless it proves insufficient — then `@dnd-kit`).

7 Testing

- Unit: slug generation, ordering logic, status transitions, category-delete guard.
- Component: product card, grid filtering, editor field→destination labels.
- RLS: impersonation checklist (anon sees only published; admin sees all; anon writes fail).
- Manual: draft → preview → publish → appears publicly; reorder reflects on `/products`; unpublish 404s the detail page.

8 Build order (three PRs)

1. `feat/backend-foundation` — migrations (tables, indexes, RLS, seed categories + demo product), typed Supabase clients (server/browser), admin auth (`/admin/login`, guarded layout, sign-out).
2. `feat/products-public` — `/products` grid + `/products/[slug]` detail from real data, SEO, order-form CTA hookup, empty states.
3. `feat/products-admin` — CRUD editor, draft/preview/publish, drag reorder, categories manager, image uploads, revalidation.

Each PR ships the full green bar (typecheck, lint, tests, build) + screenshots before merge.